# Machine Learning Systems and Hardware (COMP70114)

Coursework: CUDA Kernel Development Report

Anas Khan

November 27, 2025

## 1 Part 1: Batch Normalization Inference

**Performance:** Time Cost **0.981 ms**, passing **4/4** test cases (Submission #3869).

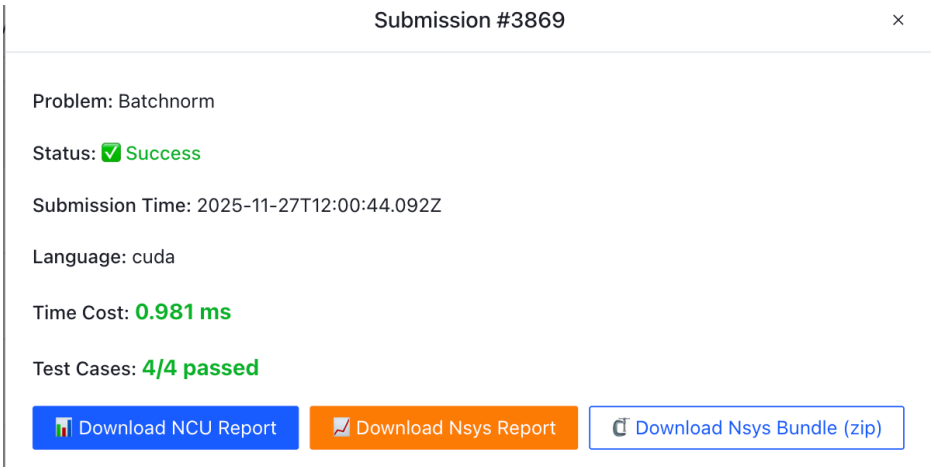



Figure 1: Batch Normalization results.

### Kernel Design and Optimizations

**One-Block-Per-Channel Architecture:** Grid dimensions `(C,1,1)` map each block to one channel. Each block independently computes $\mu_c$ and $\sigma_c^2$ across all spatial ($H \times W$) and batch ($N$) elements, eliminating inter-block synchronization.

**Vectorized Memory Access (`float4`):** 128-bit transactions load 4 floats per memory operation, achieving optimal coalescing:

```
const float4* src4 = (const float4*)src;
float4 v = src4[i];  // Single 128-bit coalesced load
local_sum += v.x + v.y + v.z + v.w;
local_sq  += v.x*v.x + v.y*v.y + v.z*v.z + v.w*v.w;
```

A cleanup loop handles remainders when $H \times W \bmod 4 \neq 0$.

**Two-Level Parallel Reduction:**

- *Warp-level:* `__shfl_down_sync()` reduces within each 32-thread warp without shared memory, eliminating bank conflicts.
- *Block-level:* Only 32 floats of shared memory store warp partial sums, which are then reduced by warp 0.

**Register-Level Accumulation:** Partial sums (`local_sum`, `local_sq`) accumulate in registers throughout the spatial/batch loops, avoiding shared memory traffic.

**Fused Scale-Shift Transformation:** The output normalization $y = \gamma \frac{x-\mu}{\sqrt{\sigma^2+\varepsilon}} + \beta$ is algebraically rewritten as a single FMA: $y = \text{scale} \cdot x + \text{shift}$, where both constants are precomputed by thread 0 using `rsqrtf()` for fast reciprocal square root.

## 2 Part 2: Depthwise 2D Convolution

**Performance:** Time Cost **0.098 ms**, passing **5/5** test cases (Submission #3747).

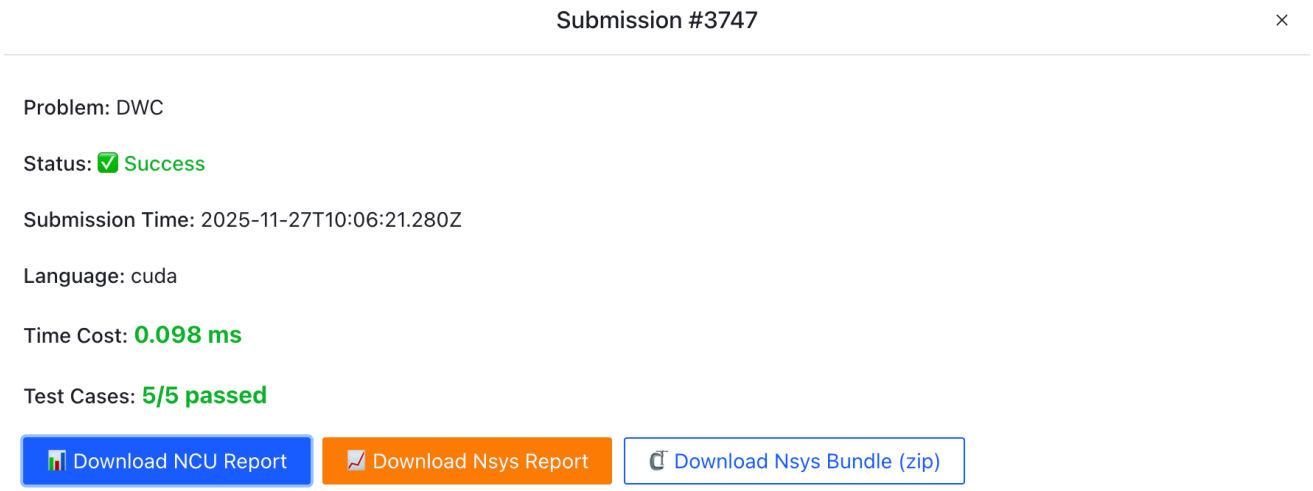



Figure 2: Depthwise Convolution results.

### Kernel Design and Optimizations

**One-Thread-Per-Output-Pixel Strategy:** Each thread computes exactly one $y[n, c, oh, ow]$, ensuring race-free execution without atomics. The 3D grid `(spatial_blocks, C, N)` naturally maps to NCHW layout.

**Memory Coalescing via Spatial Indexing:** The $x$-dimension maps to output width ($OW$), so consecutive threads access consecutive input addresses when `stride=1`: threads read `x[...+(ow*stride+kw)]`, yielding coalesced 128B transactions.

**Implicit Zero-Padding:** Rather than allocating padded input, boundary checks (`ih` $\geq$ `0 && ih < H && iw` $\geq$ `0 && iw < W`) implicitly treat out-of-bounds as zero, saving memory and avoiding copy overhead.

**Register-Based Convolution:** The $K_H \times K_W$ convolution accumulates in a register variable (`sum`), with a single global memory write per output pixel.